

DPDP COMPLIANCE AUDIT

medusa

Digital Personal Data Protection Act 2023



Grade F — Critical

Scanned: 13 April 2026

Files indexed: 10419

CONFIDENTIAL — Internal Use Only

DPDP Compliance Scan Report

Quick Actions for Developers

Fix these first to improve your compliance score:

- **DEBUG_MODE_ENABLED** → packages/medusa/src/commands/develop.ts
- **CROSS_BORDER_TRANSFER_RISK** → ...product-import/helpers/import-template.ts
- **INCOMPLETE_DELETION_COVERAGE** (codebase-wide)
- **NO_DELETION_MECHANISM** → ...modules/helpers/create-authenticated-customer.ts
- **RETENTION_PARTIAL** (codebase-wide)

This codebase has 10 critical compliance gap(s) that require immediate attention under the Digital Personal Data Protection Act 2023. An additional 6 significant gap(s) were identified that should be addressed within 30 days. The estimated total remediation effort is 10.1–38.9 engineering days.

Total findings: 24

- Rule Engine: 24
- AI Deep Review: 0

By Severity:

- HIGH: 10
- MEDIUM: 6
- LOW: 2
- INFO: 3
- PASS: 3

Developer Priority:

- Fix Now: 6
- Fix This Sprint: 4

Compliance score by section

DPDP Section	Weight	Score	Status
Section 6	22	0/22 (0%)	FAIL
Section 6(6)	8	8/8 (100%)	PASS
Section 7	4	4/4 (100%)	PASS
Section 8(1)	18	0/18 (0%)	FAIL
Section 8	10	0/10 (0%)	FAIL
Section 8(3)	8	4.0/8 (50%)	WARN
Section 8(4)	8	6.8/8 (85%)	PASS
Section 8(6)	8	5.6/8 (70%)	WARN
Section 9	6	6/6 (100%)	PASS
Section 11	6	6/6 (100%)	PASS
Section 16	4	0/4 (0%)	FAIL
Section 5	2	0/2 (0%)	FAIL
Section 13 — Grievance Redressal	4	0/4 (0%)	FAIL

Estimated Total Remediation Effort: 10.1–38.9 engineering days

Scan Coverage

Files analyzed: 10419

Data Flow Analysis

The following PII flow signals were inferred via static analysis (import/call adjacency). These are heuristic code-path chains, not proven runtime traces. Use them to guide developer review (confirm actual request → handler → service → database paths in your app).

Flow 1: Application Logs

PII fields: password

create-db.ts → log-message.ts

Flow 2: Application Logs

PII fields: password

create-db.ts → log-message.ts → logger.ts

Flow 3: Application Logs

PII fields: password

create.ts

Flow 4: Application Logs

PII fields: address

customer.ts → client.ts

Rule	Section	Severity	File
DEBUG_MODE_ENABLED	Section 8(1) — Security	HIGH	packages/medusa/src/commands/develop.ts
CROSS_BORDER_TRANSFER_RISK	Section 16 — Cross-Border Transfer	HIGH	...product-import/helpers/import-template.ts
INCOMPLETE_DELETION_COVERAGE	Section 8 — Data Principal Rights	HIGH	N/A
GRIEVANCE_OFFICER_MISSING	Section 13 — Grievance Redressal	HIGH	N/A
CONSENT_MISSING	Section 6 — Consent	HIGH	...[id]/@transfer-ownership/page.tsx
CONSENT_MISSING	Section 6 — Consent	HIGH	packages/admin/dashboard/src/routes/invite/invite.tsx
PASSWORD_NOT_HASHED	Section 8(1) — Security	HIGH	MULTIPLE
HARDCODED_SECRET	Section 8(1) — Security	HIGH	MULTIPLE
PLAINTEXT_PII_IN_LOGS	Section 8(1) — Security	HIGH	packages/design-system/toolbox/src/commands/tokens/command.ts

PLAINTEXT_PII_IN_LOGS	Section 8(1) — Security	HIGH	packages/medusa/src/migration-scripts/create-super-admin-role.ts
AUDIT_TRAIL_PARTIAL	Section 8(4) — Audit Trail	LOW	N/A
NO_DELETION_MECHANISM	Section 8 — Data Principal Rights	MEDIUM	integration-tests/modules/helpers/create-authenticated-customer.ts
RETENTION_PARTIAL	Section 8(3) — Retention	MEDIUM	N/A
NO_ERROR_HANDLING_IN_AUTH	Section 8(6) — Breach	MEDIUM	integration-tests/modules/helpers/create-authenticated-customer.ts
THIRD_PARTY_PII_SHARING	Section 5 — Notice	MEDIUM	packages/core/core-flows/src/order/workflows/create-fulfillment.ts
THIRD_PARTY_PII_SHARING	Section 5 — Notice	MEDIUM	packages/core/core-flows/src/order/workflows/create-order.ts
RETENTION_MISSING	Section 8(3) — Retention	MEDIUM	MULTIPLE
DATA_PORTABILITY_MISSING	Section 11 — Data Portability	LOW	N/A
INTERNAL_ROUTE_PII_ACCESS	Section 7 — Legitimate Use	INFO	packages/design-system/icons/src/components/payphone.tsx
INTERNAL_ROUTE_PII_ACCESS	Section 7 — Legitimate Use	INFO	packages/design-system/icons/src/components/credit-card.tsx
INTERNAL_ROUTE_PII_ACCESS	Section 7 — Legitimate Use	INFO	www/utills/packages/utills/src/examples-utills.ts
CONSENT_WITHDRAWAL_PRESENT	Section 6(6) — Consent Withdrawal	PASS	...workflow-execution-timeline-section/workflow-execution-timeline-section.tsx
BREACH_INDICATORS_ADEQUATE	Section 8(6) — Breach	PASS	N/A
DATA_ACCESS_PRESENT	Section 11 — Right to Access	PASS	packages/medusa/src/api/store/customers/middlewares.ts

5 finding(s) show rule-engine fallback text only (LLM enrichment cap or parse failure).

30-Day Remediation Roadmap

The following plan prioritises findings by severity and estimated effort. Complete Week 1 items before your next security review.

Week 1 — Critical (23–85 hours)

Fix immediately — these represent the highest legal exposure.

- Debug Mode Enabled — `commands/develop.ts` (4–16 hrs)
- Cross Border Transfer Risk — `helpers/import-template.ts` (4–12 hrs)
- Incomplete Deletion Coverage (4–16 hrs)
- Grievance Officer Missing (1–4 hrs)
- Consent Missing — `@transfer-ownership/page.tsx` (2–8 hrs)
- Consent Missing — `invite/invite.tsx` (2–8 hrs)
- Password Not Hashed — MULTIPLE (2–6 hrs)
- Hardcoded Secret — MULTIPLE (2–9 hrs)
- Plaintext Pii In Logs — `tokens/command.ts` (1–3 hrs)
- Plaintext Pii In Logs — `migration-scripts/create-super-admin-role.ts` (1–3 hrs)

Week 2 — High Priority (9–29 hours)

Address within 2 weeks — short fixes with significant compliance impact.

- No Deletion Mechanism — `helpers/create-authenticated-customer.ts` (2–6 hrs)
- Retention Partial (2–8 hrs)
- No Error Handling In Auth — `helpers/create-authenticated-customer.ts` (1–3 hrs)
- Third Party Pii Sharing — `workflows/create-fulfillment.ts` (2–6 hrs)
- Third Party Pii Sharing — `workflows/create-order.ts` (2–6 hrs)

Week 3 — Significant (45–181 hours)

Schedule in sprint — requires more engineering time.

- Retention Missing — MULTIPLE (45–181 hrs)

Week 4 — Advisory (4–16 hours)

Address before next compliance review.

- Audit Trail Partial (2–8 hrs)
- Data Portability Missing (2–8 hrs)

Total estimated remediation effort: 81–311 hours (10.1–38.9 engineering days)

Changes Since 2026-04-13

✓ 2 finding(s) resolved
— 24 finding(s) unchanged

Resolved since last scan:

- ✓ Deep Review Validated
- ✓ Deep Review Validated

Details

Finding 1 of 24 — ■ FIX THIS SPRINT

[Section 8(1) — Security]

1. [HIGH] DEBUG_MODE_ENABLED — packages / medusa / src / commands / develop.ts:82**Evidence snippet:** NODE_ENV: "development",**Estimated fix time:** 4–16 hours

Debug mode is enabled. In production, this exposes interactive debuggers (Flask/Werkzeug), detailed stack traces, and internal system information to users. Flask debug mode specifically allows arbitrary code execution via the Werkzeug console. DPDP Section 8(1) requires appropriate security measures to protect personal data.

Risk:

Manual review required. Rule 'DEBUG_MODE_ENABLED' fired on packages/medusa/src/commands/develop.ts — see rule description for details.

Fix:

1. Review the finding description above and apply fixes per your compliance process.

DPDP:

Section 8(1) — Security

Finding 2 of 24 — ■ FIX THIS SPRINT

[Section 16 — Cross-Border Transfer]

2. [HIGH] CROSS_BORDER_TRANSFER_RISK — packages / admin / dashboard / src / routes / products / product-import / helpers / import-template.ts:4

Evidence snippet: ,coffee-mug-v3,Medusa Coffee Mug,,Every programmer's best friend.,published,https://medusa-public-images.s3.eu-west-1.amazonaws.com/coffee-mug.png,400,,,,,,,,,TRUE,,,One Size,,,FALSE,TRUE,,,,,,,,

Estimated fix time: 4–12 hours

Foreign cloud storage/CDN detected with PII models in repo. Cross-border transfer risk.

Risk:

Manual review required. Rule 'CROSS_BORDER_TRANSFER_RISK' fired on packages/admin/dashboard/src/routes/products/product-import/helpers/import-template.ts — see rule description for details.

Fix:

1. Review the finding description above and apply fixes per your compliance process.

DPDP:

Section 16 — Cross-Border Transfer

Finding 3 of 24 — ■ FIX THIS SPRINT

[Section 8 — Data Principal Rights]

3. [HIGH] INCOMPLETE_DELETION_COVERAGE — N / A**Estimated fix time:** 4–16 hours

Deletion mechanism detected but may not cover all PII storage. Coverage: 24% of inferred PII entities (86/359 including FK dependents). Uncovered: account_transaction, admin_views_entity_column, admin_views_entity_configuration, admin_views_entity_configurations_active, admin_views_entity_configurations_id. DPDP Section 8 requires complete erasure across all systems.

Entity deletion coverage: 24% (86 covered / 273 uncovered)**Uncovered entities:** account_transaction, admin_views_entity_column, admin_views_entity_configuration, admin_views_entity_configurations_active, admin_views_entity_configurations_id, base_repository_find, campaign_budget, campaign_budget_usage, and 265 more**Entity→file hints:** account_transaction: models/account-transaction.ts;
admin_views_entity_column: paths/admin_views_{entity}_columns.yaml;
admin_views_entity_configuration: paths/admin_views_{entity}_configurations.yaml;
admin_views_entity_configurations_active:
paths/admin_views_{entity}_configurations_active.yaml; admin_views_entity_configurations_id:
paths/admin_views_{entity}_configurations_{id}.yaml**Risk:**

Manual review required. Rule 'INCOMPLETE_DELETION_COVERAGE' fired on see finding scope — see rule description for details.

Fix:

1. Review the finding description above and apply fixes per your compliance process.

DPDP:

Section 8 — Data Principal Rights

Finding 4 of 24 — ■ FIX THIS SPRINT

[Section 13 — Grievance Redressal]

4. [HIGH] GRIEVANCE_OFFICER_MISSING — N / A**Estimated fix time:** 1–4 hours

No compliant Grievance Officer / DPO publication found. A PASS requires grievance-related text or a dedicated route **and** a contact email or dedicated grievance/DPO URL. Tiny repos without a grievance route are not marked PASS.

Risk:

Manual review required. Rule 'GRIEVANCE_OFFICER_MISSING' fired on see finding scope — see rule description for details.

Fix:

1. Review the finding description above and apply fixes per your compliance process.

DPDP:

Section 13 — Grievance Redressal

Finding 5 of 24 — ■ FIX NOW

[Section 6 — Consent]

5. [HIGH] CONSENT_MISSING — packages / plugins / draft-order / src / admin / routes / draft-orders / [id] / @transfer-ownership / page.tsx:70

Evidence snippet: ? `\${name} (\${order.customer.email})`

Estimated fix time: 2–8 hours

Personal data collection and API endpoint detected with no consent mechanism.

Risk:

The application collects and displays customer email addresses without a clear consent mechanism. This violates the DPDP Act's requirement for explicit consent before processing personal data, potentially leading to unauthorized data use and legal penalties.

Fix:

1. Implement a consent banner or checkbox on relevant user-facing forms (if applicable) to obtain explicit user consent before collecting or displaying email addresses.
2. For internal views like this admin route, document the legal basis for processing (e.g., legitimate interest for order management) and ensure access is restricted to authorized personnel.
3. If this is an internal audit/compliance view, verify that access is restricted to authorized staff only and the purpose is documented. If so, no consent mechanism is required for this processing under Section 7.
4. Review the `useUpdateDraftOrder` hook and ensure that any data passed to it is handled in accordance with consent policies.

DPDP:

Section 6(1) — Consent requires explicit, informed, and freely given consent for the processing of personal data.

Example:

```
import React from 'react'; const ConsentCheckbox = ({ onConsentChange }) => { return ( <div>  
<input type='checkbox' id='consent' onChange={onConsentChange} /> <label htmlFor='consent'>I  
consent to the processing of my personal data.</label> </div> ); }; export default  
ConsentCheckbox;
```


Finding 6 of 24 — ■ FIX NOW

[Section 6 — Consent]

6. [HIGH] CONSENT_MISSING — packages / admin / dashboard / src / routes / invite / invite.tsx:13**Evidence snippet:** import { useSignUpWithEmailPass } from "../hooks/api/auth"**Estimated fix time:** 2–8 hours

Personal data collection and API endpoint detected with no consent mechanism.

Risk:

The application collects email and password information during the invite/signup process without a clear consent mechanism. This violates the DPDP Act's requirement for explicit consent before processing sensitive personal data, potentially leading to unauthorized data use and legal penalties.

Fix:

1. Implement a consent checkbox within the invite/signup form, requiring users to explicitly agree to terms and privacy policy before account creation.
2. Ensure the consent text clearly explains what data is collected (email, password) and how it will be used.
3. Verify that the `useSignUpWithEmailPass` hook correctly handles user consent status before proceeding with the signup.
4. Add a link to the privacy policy near the consent checkbox.

DPDP:

Section 6(1) — Consent requires explicit, informed, and freely given consent for the processing of personal data.

Example:

```
import React, { useState } from 'react'; const InviteForm = () => { const [agreedToTerms,
setAgreedToTerms] = useState(false); const handleInviteSubmit = (event) => {
event.preventDefault(); if (!agreedToTerms) { alert('Please agree to the terms and
conditions.');
```

`return; } // Proceed with invite submission }; return (<form
onSubmit={handleInviteSubmit}> {/* ... other form fields ... */} <div> <input type='checkbox'
id='terms' checked={agreedToTerms} onChange={(e) => setAgreedToTerms(e.target.checked)} />
<label htmlFor='terms'>I agree to the Terms and Conditions and Privacy Policy.</label> </div> <button type='submit'
disabled={!agreedToTerms}>Invite</button> </form>); };`

Finding 7 of 24 — ■ FIX NOW

[Section 8(1) — Security]

7. [HIGH] PASSWORD_NOT_HASHED — MULTIPLE:33

Evidence snippet: The `useSignUpWithEmailPass` hook is defined, which is likely used for user registration and password handling.

Estimated fix time: 2–6 hours

Affected files (4):

- packages/cli/create-medusa-app/src/utls/create-db.ts
- packages/core/utls/src/common/handle-postgres-database-error.ts
- packages/core/utls/src/modules-sdk/load-module-database-config.ts
- packages/medusa/src/commands/db/create.ts

Model file has password field but no hash/bcrypt/argon/pbkdf2/scrypt/encrypt reference.

Detected in 4 files: create-db.ts, handle-postgres-database-error.ts, load-module-database-config.ts, create.ts

Risk:

The codebase lacks any evidence of password hashing or encryption before storage or transmission. Storing passwords in plain text makes them vulnerable to breaches, violating the DPDP Act's security requirements and exposing users to identity theft and fraud.

Fix:

1. Install a secure password hashing library like `bcrypt` or `argon2`.
2. In the API endpoint or service responsible for user registration (e.g., where `useSignUpWithEmailPass` is called), hash the password using the chosen library before storing it in the database.
3. Ensure a unique salt is generated for each password before hashing.
4. Update any database schemas or models to remove plaintext password fields and ensure they are handled securely.

DPDP:

Section 8(1) — Data fiduciaries must implement reasonable security safeguards to protect personal data against unauthorized access, disclosure, alteration, or destruction.

Example:

```
import bcrypt from 'bcrypt'; const SALT_ROUNDS = 10; async function hashPassword(password) {
  const salt = await bcrypt.genSalt(SALT_ROUNDS); return await bcrypt.hash(password, salt); } //
Example usage in a registration service: // const hashedPassword = await
hashPassword(plainPassword); // await userService.createUser({ email, hashedPassword });
```

Finding 8 of 24 — ■ FIX NOW

[Section 8(1) — Security]

8. [HIGH] HARDCODED_SECRET — MULTIPLE:37**Evidence snippet:** * password: "supersecret"**Estimated fix time:** 2–9 hours**Affected files (13):**

- packages/core/js-sdk/src/admin/invite.ts
- packages/core/js-sdk/src/auth/index.ts
- www/utls/generated/oas-output/operations/admin/post_admin_invites_accept.ts
- www/utls/generated/oas-output/operations/admin/post_auth_[actor_type]_[auth_provider].ts
- www/utls/generated/oas-output/operations/admin/post_auth_[actor_type]_[auth_provider]_register.ts
- www/utls/generated/oas-output/operations/admin/post_auth_[actor_type]_[auth_provider]_update.ts
- ... and 7 more files

Hardcoded hardcoded_password detected. Detected in 13 files: invite.ts, index.ts, post_admin_invites_accept.ts, post_auth_[actor_type]_[auth_provider].ts and 9 more

Risk:

Sensitive secrets, such as the JWT passphrase, are hardcoded directly in the configuration files. This increases the risk of unauthorized access if the codebase is compromised, potentially leading to session hijacking and unauthorized data access.

Fix:

1. Remove hardcoded secrets from configuration files and use environment variables or a dedicated secrets management system (e.g., HashiCorp Vault, AWS Secrets Manager).
2. Update `medusa-config.js` to read the JWT passphrase from an environment variable (e.g., `process.env.JWT\_PASSPHRASE`).
3. Ensure that the environment variable is securely set in your deployment environment.
4. Remove the hardcoded 'secret' value from the `passphrase` constant.

DPDP:

Section 8(1) — Data fiduciaries must implement reasonable security safeguards to protect personal data against unauthorized access, disclosure, alteration, or destruction.

Example:

```
const { defineConfig, Modules } = require('@medusajs/utls'); const { generateKeyPairSync } =
require('crypto'); const passphrase = process.env.JWT_PASSPHRASE; if (!passphrase) { throw new
Error('JWT_PASSPHRASE environment variable is not set.')} } const { publicKey, privateKey } =
generateKeyPairSync('rsa', { modulusLength: 4096, publicKeyEncoding: { type: 'spki', format:
'pem' }, privateKeyEncoding: { type: 'pkcs8', format: 'pem', cipher: 'aes-256-cbc',
passphrase, }, }); module.exports = defineConfig({ projectConfig: { http: { jwtSecret: { key:
privateKey, passphrase }, jwtPublicKey: publicKey, }, }, // ... other configurations
```

Finding 9 of 24 — ■ FIX NOW

[Section 8(1) — Security]

9. [HIGH] PLAINTEXT_PII_IN_LOGS — packages / design-system / toolbox / src / commands / tokens / command.ts:181

Evidence snippet: logger.info(`Writing typography tokens to file`)

Estimated fix time: 1–3 hours

PII keyword appears in logging call. Personal data may be written to logs in plaintext.

Risk:

While the log level is set to 'error', there's no explicit mechanism shown to prevent PII from being logged if an error occurs. This could lead to sensitive personal data being exposed in logs, violating data minimization and security principles.

Fix:

1. Implement a PII detection and redaction middleware for all logging statements in the application.
2. Configure a centralized logging system that supports PII masking and access control.
3. Review all existing log aggregation and storage to ensure no PII is retained in plaintext.
4. Add specific logging configurations to redact or avoid logging sensitive fields like email, name, or address when errors occur.

DPDP:

Section 8(1) — Security: Requires appropriate technical and organizational measures to ensure the security of personal data, including protection against unauthorized or unlawful processing and against accidental loss, destruction or damage.

Example:

```
import winston from 'winston'; const logger = winston.createLogger({ level:
process.env.LOG_LEVEL || 'info', format: winston.format.combine( winston.format.errors({
stack: true }), winston.format.splat(), winston.format.json(), // Add a custom format to
redact PII winston.format.def{ transform: (log) => { if (log.message && typeof log.message ===
'string') { log.message =
log.message.replace(/\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b/g,
'REDACTED_EMAIL'); } // Add more redaction rules for other PII fields as needed return log;
} } ), transports: [ new winston.transports.Console(), // Add other transports like file, etc.
], }); export default logger;
```

Finding 10 of 24 — ■ FIX NOW

[Section 8(1) — Security]

10. [HIGH] PLAINTEXT_PII_IN_LOGS — packages / medusa / src / migration-scripts / create-super-admin-role.ts:49**Evidence snippet:** `logger.info(` ${index + 1}. ${user.email || user.id} (${user.id})`)`**Estimated fix time:** 1–3 hours

PII keyword appears in logging call. Personal data may be written to logs in plaintext.

Risk:

While the log level is set to 'error', there's no explicit mechanism shown to prevent PII from being logged if an error occurs. This could lead to sensitive personal data being exposed in logs, violating data minimization and security principles.

Fix:

1. Implement a PII detection and redaction middleware for all logging statements in the application.
2. Configure a centralized logging system that supports PII masking and access control.
3. Review all existing log aggregation and storage to ensure no PII is retained in plaintext.
4. Add specific logging configurations to redact or avoid logging sensitive fields like email, name, or address when errors occur.

DPDP:

Section 8(1) — Security: Requires appropriate technical and organizational measures to ensure the security of personal data, including protection against unauthorized or unlawful processing and against accidental loss, destruction or damage.

Example:

```
import winston from 'winston'; const logger = winston.createLogger({ level:
process.env.LOG_LEVEL || 'info', format: winston.format.combine( winston.format.errors({
stack: true }), winston.format.splat(), winston.format.json(), // Add a custom format to
redact PII winston.format.def{ transform: (log) => { if (log.message && typeof log.message ===
'string') { log.message =
log.message.replace(/\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b/g,
'[REDACTED_EMAIL]'); } // Add more redaction rules for other PII fields as needed return log;
} } ), transports: [ new winston.transports.Console(), // Add other transports like file, etc.
], }); export default logger;
```

Finding 11 of 24 — ■ BACKLOG

[Section 8(4) — Audit Trail]

11. [LOW] AUDIT_TRAIL_PARTIAL — N / A**Estimated fix time:** 2–8 hours

Partial audit trail detected. Found: change tracking. Missing: access logging, audit model.

Risk:

The absence of a comprehensive audit trail means there is no record of who accessed or modified personal data, making it impossible to demonstrate compliance or investigate data breaches. This lack of accountability can lead to significant legal and reputational damage.

Fix:

1. Implement a centralized audit logging mechanism that captures all access to and modifications of personal data.
2. Define a clear audit log schema including user ID, timestamp, action performed, and data affected.
3. Integrate the audit logging mechanism into all relevant data access and modification points in the application.
4. Ensure audit logs are stored securely, are immutable, and retained for a defined period as per DPDP requirements.

DPDP:

Section 8(4) — Audit Trail: Requires the controller to maintain an audit trail of all operations performed on personal data.

Example:

```
import { Logger } from '@medusajs/utils'; const auditLogger = Logger.getLogger('audit');
export const logAudit = (userId: string, action: string, details: Record<string, any>) => {
  auditLogger.info(JSON.stringify({ userId, timestamp: new Date().toISOString(), action,
  details, })); }; // Example usage in a service: // import { logAudit } from './audit-logger';
// logAudit('user-123', 'update_profile', { field: 'email', oldValue: 'old@example.com',
newValue: 'new@example.com' });
```

Finding 12 of 24 — ■ REVIEW & FIX

[Section 8 — Data Principal Rights]

12. [MEDIUM] NO_DELETION_MECHANISM — integration-tests / modules / helpers / create-authenticated-customer.ts**Estimated fix time:** 2–6 hours

Deletion logic exists but only in internal/non-user-facing routes. Data principals need a way to request erasure (e.g. account settings, DELETE /me or /account).

Risk:

The absence of a user-facing data deletion mechanism means data principals cannot exercise their right to erasure, leading to non-compliance and potential penalties. This prevents users from controlling their personal data as mandated by the DPDP.

Fix:

1. Implement a user-facing endpoint (e.g., DELETE /me or DELETE /account) that allows data principals to request account and data deletion.
2. Create a corresponding service or controller function to handle the deletion request, ensuring all associated personal data is removed or anonymized.
3. Add a confirmation step for the user before initiating the deletion process to prevent accidental data loss.
4. Ensure that the deletion process is logged for audit purposes.

DPDP:

Section 8 — Data Principal Rights: Mandates that data principals have the right to request erasure of their personal data.

Example:

```
import { Router } from 'express'; import { CustomerService } from
'../services/customer.service'; // Assuming you have a CustomerService const router =
Router(); const customerService = new CustomerService(); router.delete('/me', async (req, res)
=> { const customerId = req.user.id; // Assuming user ID is available from authentication
middleware try { await customerService.deleteCustomerData(customerId); res.status(204).send();
// No Content } catch (error) { res.status(500).json({ message: 'Error deleting customer data'
}); } }); export default router;
```

Finding 13 of 24 — ■ REVIEW & FIX

[Section 8(3) — Retention]

13. [MEDIUM] RETENTION_PARTIAL — N / A**Estimated fix time:** 2–8 hours

Retention signals found in 166 model(s) but 73 PII model(s) have no retention policy.
Uncovered: address.ts, AdminCreateFulfillment.yaml, AdminCustomerAddress.yaml.

Risk:

Authentication-related API calls are not wrapped in error handling, meaning that if an error occurs during these operations, it may not be logged or handled gracefully, potentially leading to unaddressed security vulnerabilities or service disruptions.

Fix:

1. Wrap the `api.post` calls in `integration-tests/modules/helpers/create-authenticated-customer.ts` within `try...catch` blocks to handle potential errors during authentication.
2. Inside the `catch` blocks, log the error details using a suitable logging mechanism.
3. Re-throw the error or return a specific error response to indicate the failure of the authentication process.

DPDP:

Section 8(6) — Breach

Example:

```
import { CreateCustomerDTO } from "@medusajs/types" export const createAuthenticatedCustomer =
async ( api: any, storeHeaders: Record<any, any>, customerData: Partial<CreateCustomerDTO> =
{} ) => { const email = customerData.email ?? "tony@start.com" try { const signup = await
api.post(REDACTED_POSSIBLE_SECRET, { email, password = "REDACTED_CREDENTIAL", }) // ... rest
of the function } catch (error) { console.error("Error during signup:", error) throw error //
or handle appropriately } }
```


Finding 14 of 24 — ■ REVIEW & FIX

[Section 8(6) — Breach]

14. [MEDIUM] NO_ERROR_HANDLING_IN_AUTH — integration-tests / modules / helpers / create-authenticated-customer.ts:245

Evidence snippet: Accesses 'order.shipping_address', which is considered personal data.

Estimated fix time: 1–3 hours

Auth routes have no try/catch or try/except blocks. Errors may go unlogged.

Risk:

Personal data, specifically shipping address details, is being processed and potentially shared with inventory management systems without explicit notice to the user, violating DPDP Section 5 requirements for transparency.

Fix:

1. Review the 'adjustInventoryLevelsStep' and any other third-party integrations to ensure they do not receive or store PII unnecessarily.
2. Implement a clear notice to users before order fulfillment that their shipping address will be used for inventory adjustments and fulfillment processing.
3. If the shipping address is not strictly required by the inventory system, remove it from the data passed to 'adjustInventoryLevelsStep'.
4. Consider data minimization techniques to only pass essential address components if the full address is required.

DPDP:

Section 5 — Notice

Example:

```
import { adjustInventoryLevelsStep } from "../../inventory" // ... other imports and types //
Example of how to potentially modify the step or data passed const fulfillmentData = { // ...
other data delivery_address: { ...shippingAddress, id: undefined } as any, // Ensure only
necessary fields are passed } // Before calling adjustInventoryLevelsStep, ensure PII is
handled or anonymized if possible // await adjustInventoryLevelsStep(fulfillmentData)
```

Finding 15 of 24 — ■ REVIEW & FIX

[Section 5 — Notice]

15. [MEDIUM] THIRD_PARTY_PII_SHARING — packages / core / core-flows / src / order / workflows / create-fulfillment.ts:245

Evidence snippet: const shippingAddress = order.shipping_address ?? { id: undefined }

Estimated fix time: 2–6 hours

Personal data present in file that imports third-party SDK. Data may be shared without user awareness or notice (DPDP Section 5).

Risk:

Personal data, including addresses and email, is processed within the order creation workflow and potentially shared with draft order adjustment processes without explicit user notice, contravening DPDP Section 5.

Fix:

1. Review the 'refreshDraftOrderAdjustmentsWorkflow' to ensure it does not unnecessarily process or store PII.
2. Implement a clear notice to users during the order creation process informing them that their address and email details will be used for order processing and draft order adjustments.
3. Apply data minimization principles to ensure only essential address and email components are passed to downstream workflows.
4. If possible, anonymize or pseudonymize address data before passing it to workflows that do not require direct PII.

DPDP:

Section 5 — Notice

Example:

```
import { refreshDraftOrderAdjustmentsWorkflow } from
".../draft-order/workflows/refresh-draft-order-adjustments" // ... other imports and types
// Example of data preparation before passing to workflows const orderData = { // ... other
data shipping_address: { ...shippingAddress, id: undefined }, // Ensure only necessary fields
billing_address: { ...billingAddress, id: undefined }, // Ensure only necessary fields email:
input.email, } // Before calling refreshDraftOrderAdjustmentsWorkflow, ensure PII is handled
or anonymized if possible // await refreshDraftOrderAdjustmentsWorkflow(orderData)
```

Finding 16 of 24 — ■ REVIEW & FIX

[Section 5 — Notice]

16. [MEDIUM] THIRD_PARTY_PII_SHARING — packages / core / core-flows / src / order / workflows / create-order.ts:62

Evidence snippet: const shippingAddress = data.input.shipping_address ?? { id: undefined }

Estimated fix time: 2–6 hours

Personal data present in file that imports third-party SDK. Data may be shared without user awareness or notice (DPDP Section 5).

Risk:

Manual review required. Rule 'THIRD_PARTY_PII_SHARING' fired on packages/core/core-flows/src/order/workflows/create-order.ts — see rule description for details.

Fix:

1. Review the finding description above and apply fixes per your compliance process.

DPDP:

Section 5 — Notice

Finding 17 of 24 — ■ REVIEW & FIX

[Section 8(3) — Retention]

17. [MEDIUM] RETENTION_MISSING — MULTIPLE

Estimated fix time: 45–181 hours

Affected files (73):

- packages/plugins/draft-order/src/admin/lib/schemas/address.ts
- www/apps/api-reference/specs/admin/components/schemas/AdminCreateFulfillment.yaml
- www/apps/api-reference/specs/admin/components/schemas/AdminCustomerAddress.yaml
- www/apps/api-reference/specs/admin/components/schemas/AdminCustomerAddressResponse.yaml
- www/apps/api-reference/specs/admin/components/schemas/AdminDraftOrder.yaml
- www/apps/api-reference/specs/admin/components/schemas/AdminDraftOrderPreview.yaml
- ... and 67 more files

Model/schema file contains PII fields but no retention or expiry signals detected. Detected in 73 files: address.ts, AdminCreateFulfillment.yaml, AdminCustomerAddress.yaml, AdminCustomerAddressResponse.yaml and 69 more

Risk:

The absence of data retention policies means personal data may be stored indefinitely, increasing the risk of unauthorized access or misuse and violating DPDP Section 8(3). This can lead to regulatory penalties and reputational damage.

Fix:

1. Implement a data retention policy for all PII fields across the application.
2. Add a 'retention_period' or 'expires_at' field to relevant data models (e.g., in database schemas or ORM configurations).
3. Develop a background job or scheduled task to identify and purge data that has exceeded its retention period.
4. Document the data retention policy and communicate it to relevant stakeholders.

DPDP:

Section 8(3) — Data Minimisation and Storage Limitation

Example:

```
from datetime import datetime, timedelta from django.db import models class
UserProfile(models.Model): email = models.EmailField() created_at =
models.DateTimeField(auto_now_add=True) # Define retention period in days RETENTION_DAYS = 365
def is_expired(self): return datetime.now() > self.created_at +
timedelta(days=self.RETENTION_DAYS) # In a management command or scheduled task: # for
user_profile in UserProfile.objects.all(): # if user_profile.is_expired(): #
user_profile.delete()
```

Finding 18 of 24 — ■ BACKLOG

[Section 11 — Data Portability]

18. [LOW] DATA_PORTABILITY_MISSING — N / A**Estimated fix time:** 2–8 hours

No data export or portability endpoint detected. DPDP Section 11 requires Data Fiduciaries to provide personal data in a commonly used, machine-readable format upon request.

Risk:

The lack of a data portability feature means users cannot easily obtain their personal data in a usable format, hindering their rights under DPDP Section 11. This can lead to user dissatisfaction and potential non-compliance penalties.

Fix:

1. Implement an API endpoint (e.g., `/users/me/data`) that allows authenticated users to request an export of their personal data.
2. Develop a service that aggregates the user's personal data from various sources within the application.
3. Format the aggregated data into a commonly used, machine-readable format (e.g., JSON, CSV).
4. Provide a mechanism for users to download the exported data (e.g., a secure link or direct download).

DPDP:

Section 11 — Data Portability

Example:

```
from fastapi import APIRouter, Depends from sqlalchemy.orm import Session from app.database import get_db from app.schemas import UserDataSchema from app.services import user_service router = APIRouter() @router.get('/users/me/data', response_model=UserDataSchema) def get_my_data(db: Session = Depends(get_db), current_user: dict = Depends(get_current_user)): user_data = user_service.get_user_data(db, current_user['id']) return user_data
```

Finding 19 of 24 — ■ BACKLOG

[Section 7 — Legitimate Use]

19. [INFO] INTERNAL_ROUTE_PII_ACCESS — packages / design-system / icons / src / components / payphone.tsx
Internal file (payphone.tsx) accesses PII. Verify access is restricted to authorised contexts and PII is not exposed to logs or error responses.

Finding 20 of 24 — ■ BACKLOG

[Section 7 — Legitimate Use]

20. [INFO] INTERNAL_ROUTE_PII_ACCESS — packages / design-system / icons / src / components / credit-card.tsx
Internal file (credit-card.tsx) accesses PII. Verify access is restricted to authorised contexts and PII is not exposed to logs or error responses.

Finding 21 of 24 — ■ BACKLOG

[Section 7 — Legitimate Use]

21. [INFO] INTERNAL_ROUTE_PII_ACCESS — www / utils / packages / utils / src / examples-utils.ts
Internal service/utility accesses PII. Verify: (1) this file is only called from authorised contexts, (2) PII is not inadvertently exposed to logs or error messages.

Finding 22 of 24 — ■ BACKLOG

[Section 6(6) — Consent Withdrawal]

22. [PASS] CONSENT_WITHDRAWAL_PRESENT — packages / admin / dashboard / src / routes / workflow-executions / workflow-execution-detail / components / workflow-execution-timeline-section / workflow-execution-timeline-section.tsx

Consent withdrawal mechanism detected. Found in: packages/admin/dashboard/src/routes/workflow-executions/workflow-execution-detail/components/workflow-execution-timeline-section/workflow-execution-timeline-section.tsx (line 105)

Finding 23 of 24 — ■ BACKLOG

[Section 8(6) — Breach]

23. [PASS] BREACH_INDICATORS_ADEQUATE — N / A

Logging and/or breach alerting present.

Finding 24 of 24 — ■ BACKLOG

[Section 11 — Right to Access]

24. [PASS] DATA_ACCESS_PRESENT — packages / medusa / src / api / store / customers / middlewares.ts

Data access endpoint detected for authenticated users.